

# Tipuri comune de complexitati si cum pot fi recunoscute rapid

## 1. $O(1)$ - Complexitate constanta:

- **Descriere:** Timpul de executie ramane constant, indiferent de dimensiunea datelor de intrare ( $n$ ).
- **Indicii in cod:**
  - Operatii simple: adunari, scaderi, inmultiri, impartiri, accesarea unui element dintr-un array prin index.
  - Nu exista bucle care sa depinda de  $n$ .
- **Exemplu:**

```
a = 5 + 3 # O(1)
element = lista[0] # O(1)
```

## 2. $O(\log n)$ - Complexitate logaritmica:

- **Descriere:** Timpul de executie creste foarte incet pe masura ce  $n$  creste. La fiecare pas, dimensiunea problemei este divizata (de obicei, la 2 sau la o alta constanta).
- **Indicii in cod:**
  - Impartirea repetata a datelor de intrare (de exemplu,  $n = n // 2$ ,  $n = n // 10$ ).
  - Cautare binara intr-o lista sortata.
  - Parcurgerea unui arbore binar echilibrat.

- **Exemplu:**

```
while n > 0:
    n = n // 2 # O(log n)
```

## 3. $O(n)$ - Complexitate liniara:

- **Descriere:** Timpul de executie creste direct proportional cu  $n$ . Daca  $n$  se dubleaza, timpul de executie se dubleaza si el.
- **Indicii in cod:**
  - O singura bucla for sau while care itereaza prin toate elementele datelor de intrare.
- **Exemplu:**

```
for i in range(n): # O(n)
    # ...
```

#### 4. $O(n \log n)$ - Complexitate liniar-logaritmică:

- **Descriere:** Reprezintă o combinație între complexitatea liniară și cea logaritmică. Este mai puțin eficientă decât  $O(n)$ , dar mai eficientă decât  $O(n^2)$ .
- **Indicii în cod:**
  - Algoritmi de sortare eficienți, cum ar fi Mergesort sau Heapsort.
  - O buclă care se execută de  $n$  ori, iar în interiorul ei se efectuează o operație cu complexitate  $O(\log n)$ .

- **Exemplu (Mergesort - idee):**

```
def mergesort(lista):
    if len(lista) > 1:
        mijloc = len(lista) // 2
        stanga = lista[:mijloc]
        dreapta = lista[mijloc:]
        mergesort(stanga) #O(log n) pași de împartire
        mergesort(dreapta) #O(log n) pași de împartire
        # ... combinare O(n)
```

#### 5. $O(n^2)$ - Complexitate pătratică:

- **Descriere:** Timpul de execuție crește proporțional cu pătratul lui  $n$ . Dacă  $n$  se dublează, timpul de execuție se înmulțește cu 4.
- **Indicii în cod:**
  - Două bucle imbricate, fiecare iterând prin datele de intrare.

- **Exemplu:**

```
for i in range(n):
    for j in range(n): # O(n^2)
        # ...
```

#### 6. $O(2^n)$ - Complexitate exponențială:

- **Descriere:** Timpul de execuție crește foarte rapid pe măsura ce  $n$  crește. Este extrem de ineficientă pentru valori mari ale lui  $n$ .
- **Indicii în cod:**
  - Recursivitate cu mai multe apeluri recursive pentru fiecare apel (de exemplu, calculul numerelor Fibonacci prin recursivitate directă).
  - Generarea tuturor submultimilor unui set.

• **Exemplu (Fibonacci recursiv - ineficient):**

```
def fibonacci(n):
    if n <= 1:
        return n
    else:
        return fibonacci(n-1) + fibonacci(n-2) # O(2^n)
```

**Tabel recapitulativ:**

| Complexitate  | Descriere          | Crestere pe masura ce n creste | Exemplu in cod                                          |
|---------------|--------------------|--------------------------------|---------------------------------------------------------|
| $O(1)$        | Constanta          | Foarte lenta (deloc)           | <code>a = b + c,</code><br><code>lista[index]</code>    |
| $O(\log n)$   | Logaritmica        | Foarte lenta                   | <code>n = n // 2,</code><br>cautare binara              |
| $O(n)$        | Liniara            | Liniara                        | <code>for i in</code><br><code>range(n):</code>         |
| $O(n \log n)$ | Liniar-logaritmica | Putin mai rapida decat $n^2$   | Mergesort,<br>Heapsort                                  |
| $O(n^2)$      | Patratica          | Rapida                         | Doua bucle for imbricate                                |
| $O(2^n)$      | Exponentiala       | Foarte rapida                  | Fibonacci recursiv (ineficient),<br>generare submultimi |

**Observatie importanta:** Acestea sunt doar niste reguli generale. Analiza exacta a complexitatii poate fi mai complexa in anumite cazuri, dar aceste indicii pot oferi o estimare rapida in majoritatea situatiilor.

# Diagrama vizuala complexitati

